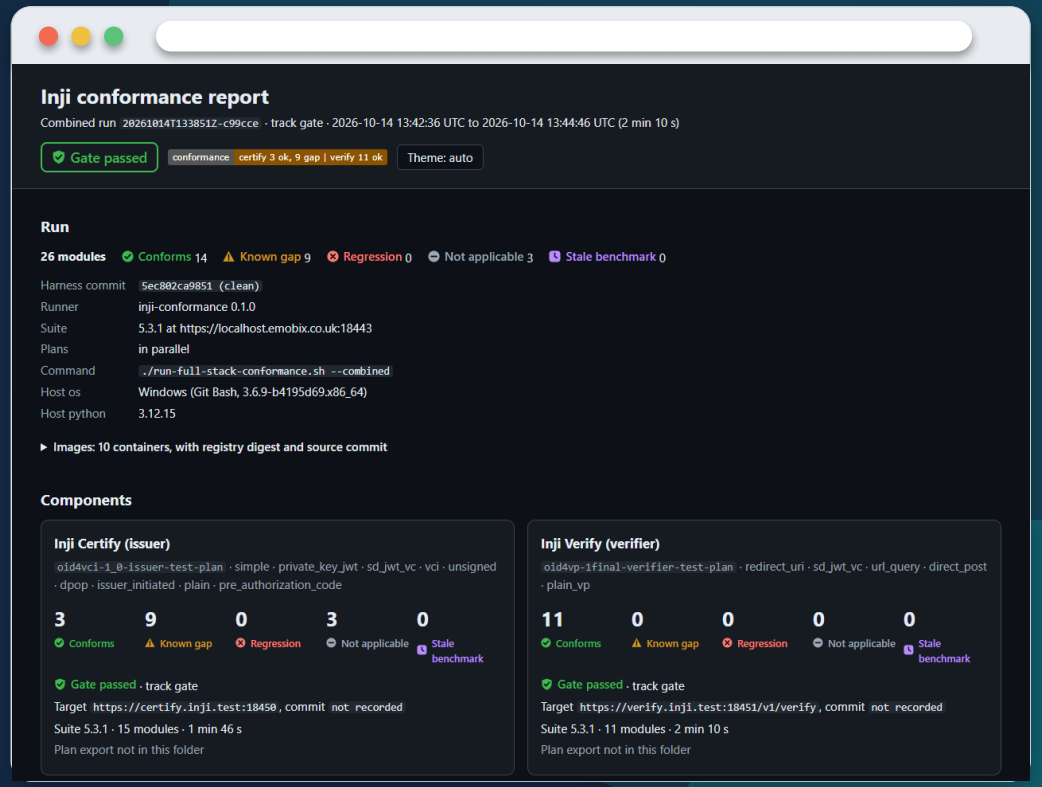
MOSIP DECODE 2026 · PROBLEM STATEMENT 01

Inji conformance harness

Automated OpenID conformance testing for Inji Certify and Inji Verify, with no person in the loop



Team Badmosip

Kanav Bhardwaj · Ivan Bhargava · Nikunj Mahajan
Ishaan Sodhi · Harsh Modi

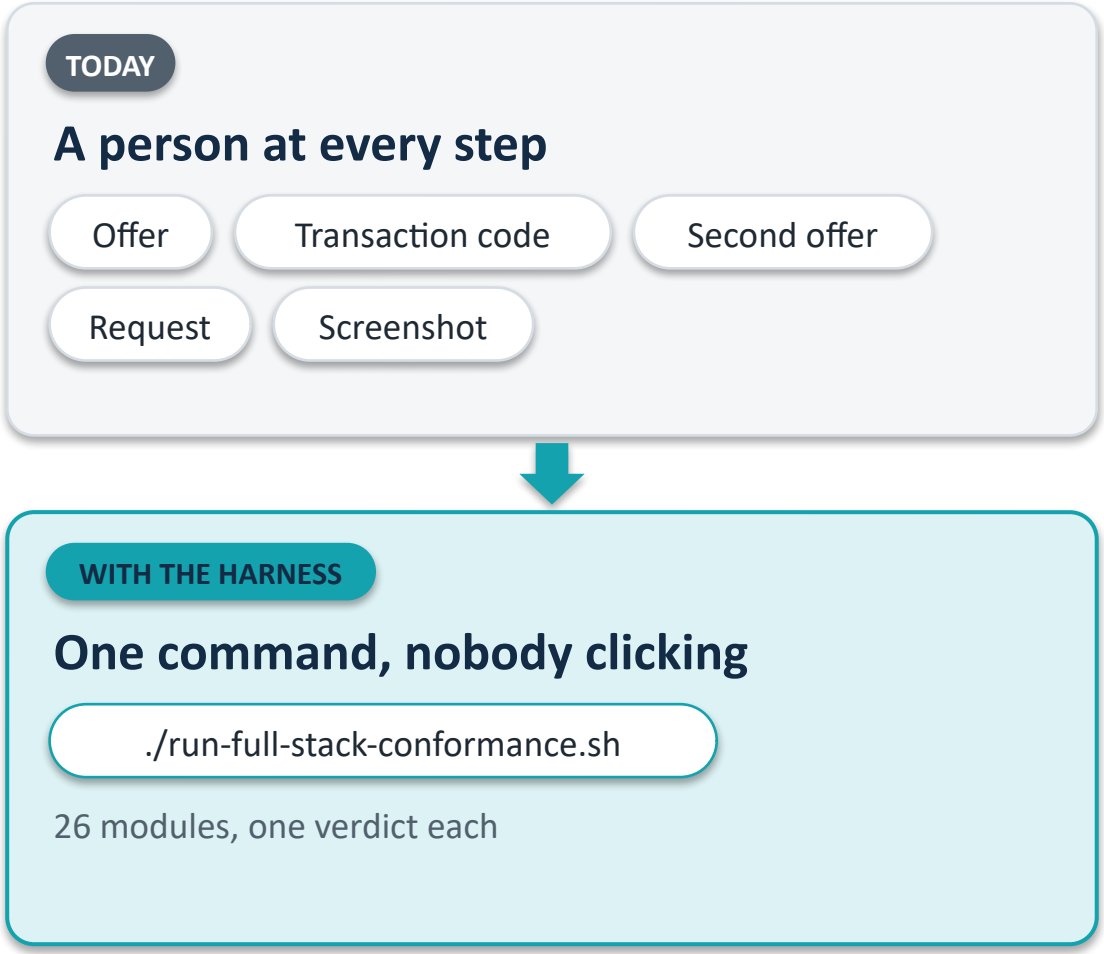
Conformance testing needs a person at every step

1. **The suite waits for a person**
   offers, transaction codes, requests, screenshots

2. **Results stay outside Inji's test reports**
   no rows in the api-testrigs the Inji teams read

3. **No gate in CI**
   a release can break conformance without anyone seeing it

4. **Blockers stay hidden**
   as served, 13 of 15 issuer modules stop at the first check



TARGET AUDIENCE

Built for the teams that ship and certify Inji



Inji release teams

find conformance regressions at release time



Countries and integrators

need certifiable OpenID issuers and verifiers



QA teams

want conformance rows in their api-testrig reports

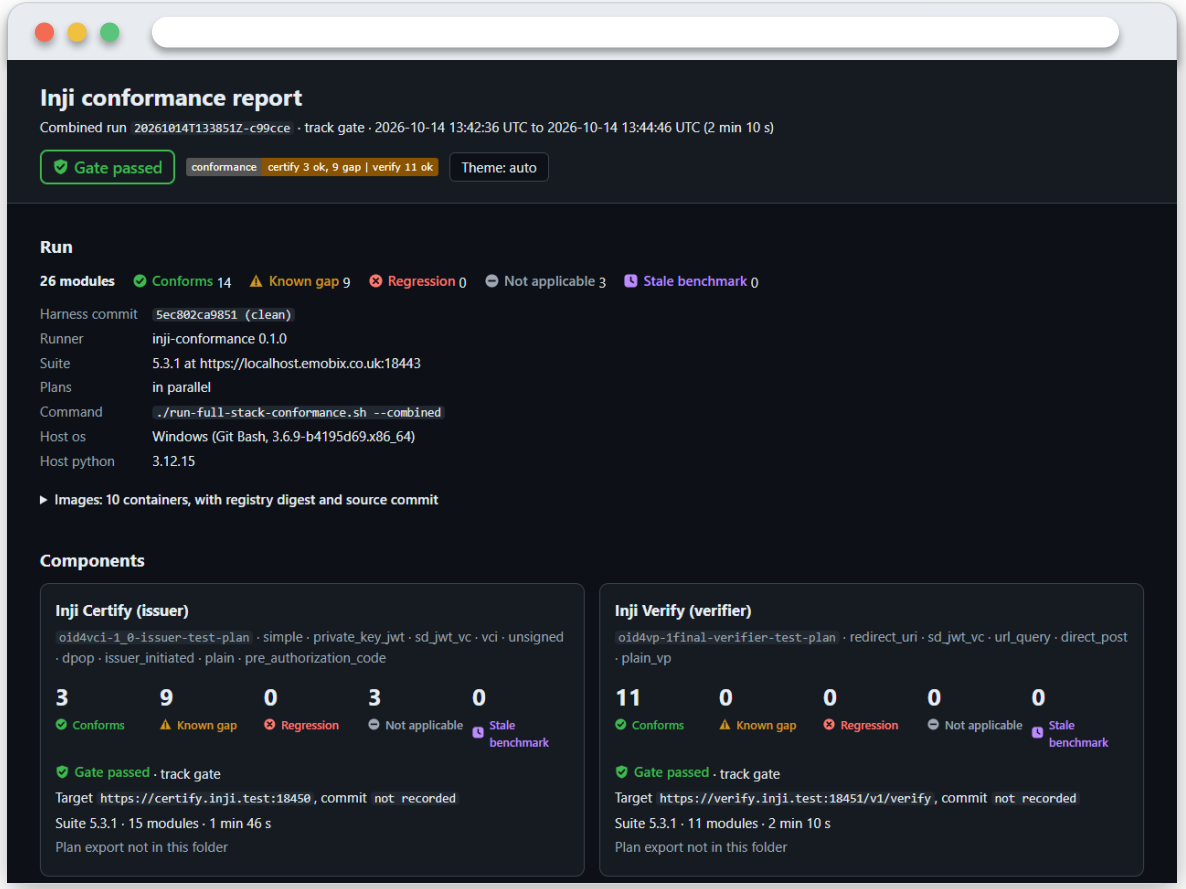
One command runs 26 modules with no one clicking

1. **./run-full-stack-conformance.sh**
   starts the suite, Inji Certify, Inji Verify and MinIO

2. **Handoff responder**
   answers every step where the suite waits

3. **Benchmark gate**
   one verdict per module; a regression fails the build

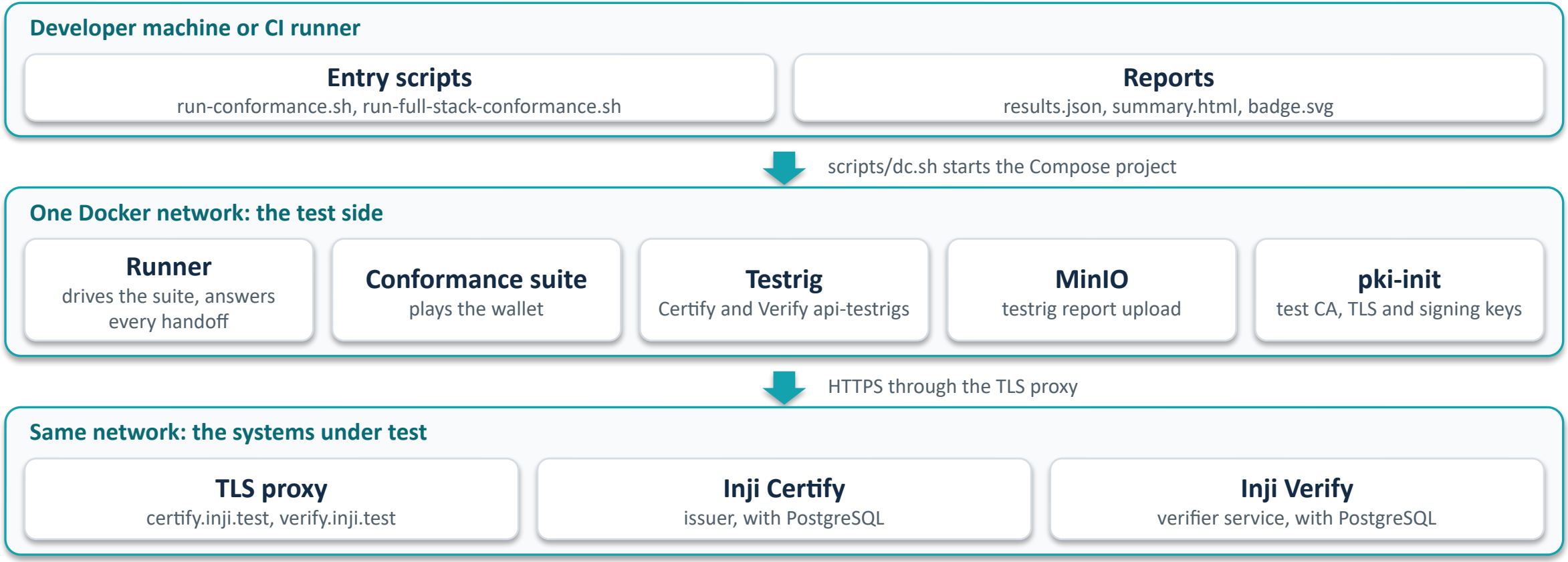
4. **Reports**
   HTML report, badge, TestNG rows in both api-testrigs



Consolidated report of a full-stack run

ARCHITECTURE

Everything runs in one Docker Compose project



GitHub Actions runs the same scripts on push, on release, weekly, and from Inji release pipelines.

TECHNOLOGY STACK

Python and Bash drive the suite; Java feeds the api-testrigs

01 Languages

- Python 3.12: runner, gate, reports
- Bash: run scripts
- Java 21: api-testrig bridge module

02 Libraries and tools

- httpx, Jinja2, jsonschema, Pillow
- pytest, ruff, shellcheck
- TestNG and Maven in the api-testrigs

03 MOSIP and Inji

- Inji Certify (issuer)
- Inji Verify (verifier)
- Inji api-testrigs and their S3 report upload

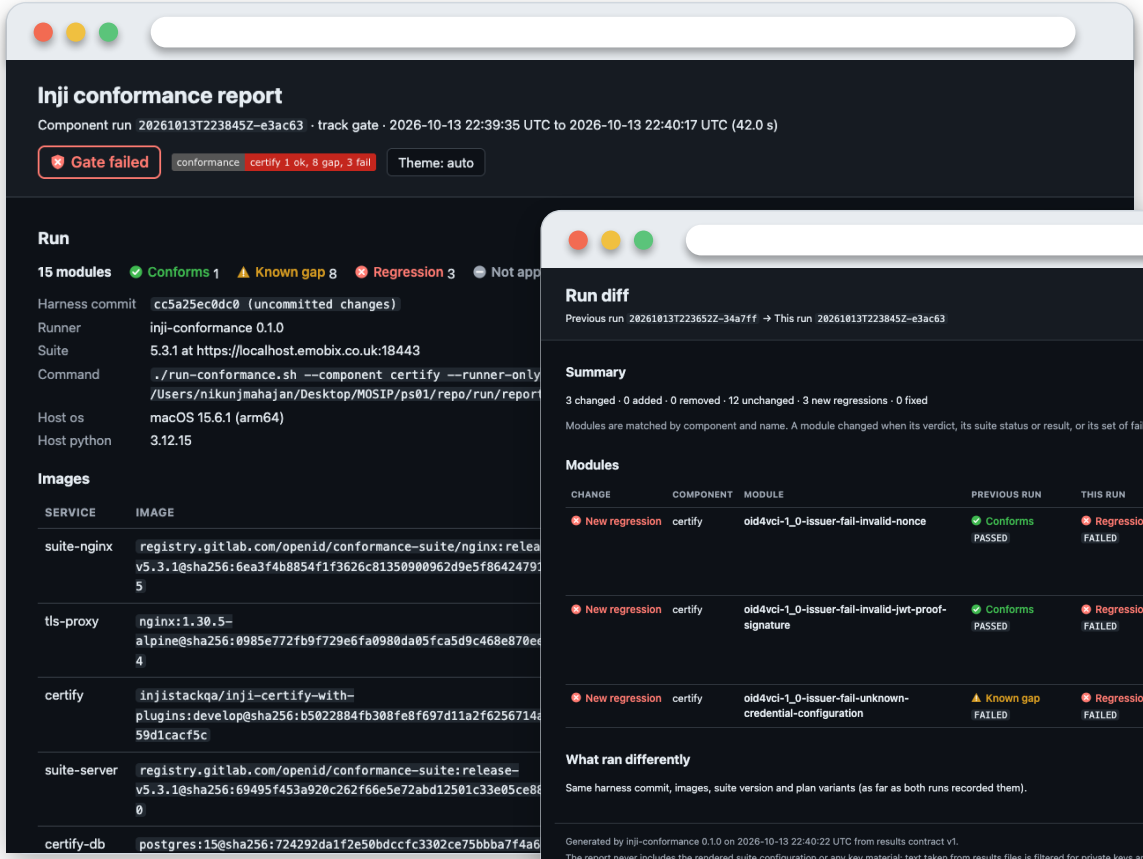
04 Infrastructure

- OpenID conformance suite release-v5.3.1
- Docker Compose v2, PostgreSQL, MongoDB, nginx TLS proxy
- MinIO (S3), GitHub Actions

Break Certify on purpose and the gate fails the build

1. **Start**
   one command, both plans in parallel
2. **Handoffs**
   each log line is one answered step
3. **Report**
   one verdict per module
4. **Testrigs**
   TestNG rows uploaded to MinIO
5. **Break it**
   demo.sh --regression edits Certify
6. **Gate**
   exit 1; the diff names the three modules

Demo video:
https://youtu.be/mlXXie1bCXY



Gate failed: 3 regressions

Run diff names each module

EVIDENCE

26 modules reach a verdict and none regress

11 / 11

Inji Verify modules conform

7 of them judged from Inji Verify's API verdict

15 / 15

Inji Certify modules reach a verdict

3 pass, 9 known Inji gaps, 3 not applicable

0

regressions

both gates passed

728 s

full stack from a fresh clone

Windows 11, with both api-testrigs and MinIO

Also measured: 8 green hosted GitHub Actions runs on main (2026-10-14 to 2026-10-24) · 472 runner unit tests passed (pytest, 2026-10-25) · Run ids and evidence folders in docs/test-report.md.

REQUIREMENTS COVERED

Every mandatory and good-to-have task has a file behind it

Mandatory tasks

Docker Compose stack	docker-compose.yml
Programmatic runner	runner/
api-testrig integration	testrigs/
Combined run, one report	run-conformance.sh --combined
One command per component	run-conformance.sh --component

Good-to-have and bonus

GitHub Actions gate	.github/workflows/
Selective execution	docs/api-automation.md
Expected failures	docs/benchmark.md
Result diff between runs	inji-conformance diff
Documentation	docs/README.md
Bonus: parallel execution	docs/test-report.md

Tick: built and shown by a run record or file in the repository. Source: docs/submission-checklist.md.

IMPACT

Release teams get a CI gate; QA teams get rows in their own reports

Who benefits



Inji release teams
a CI gate on every release



QA teams
conformance rows next to their API tests



Integrators
a ranked HAIP readiness list

What changes, counted

5

handoff kinds automated

offer, code, second offer, request, screenshot

26

modules per command

15 Certify and 11 Verify

26

TestNG rows per run

in two api-testrig reports

31-39 s

plans run in parallel

81-101 s one after the other; Docker with 12 CPUs

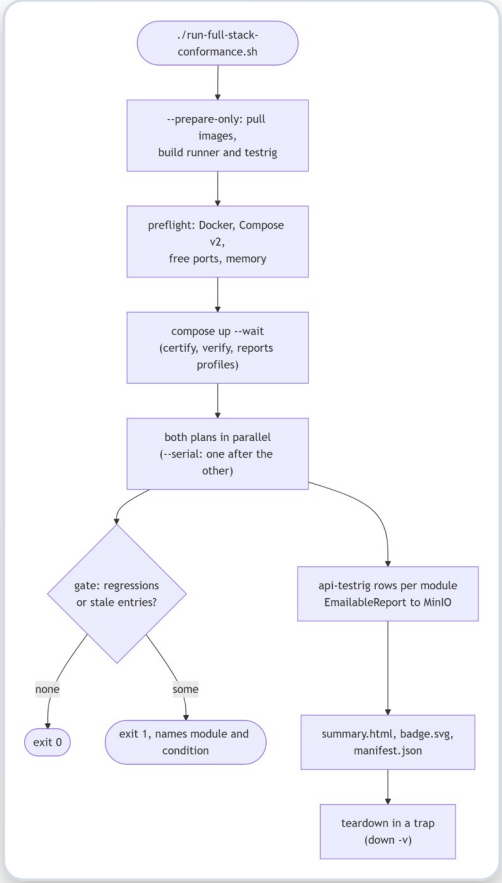
Results land inside Inji's own tools, with no person in the loop

1. **No person in the loop**
   Inji Verify's own verdict is uploaded as the screenshot

2. **Every accepted failure is written down**
   owner, spec clause and finding per entry

3. **Results inside Inji's own tools**
   api-testrig rows and MinIO reports

4. **Callable from Inji release pipelines**
   workflow_call fails a release on regression



User flow of one run

Scope and next steps

Scope

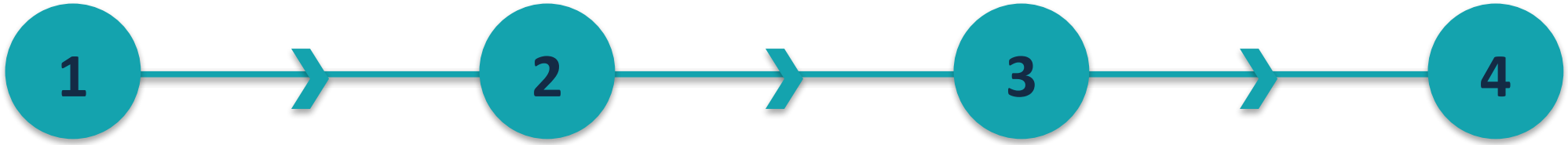
- Certify's metadata lacks authorization_endpoint; the TLS proxy adds it, and the gate, report and badge say so
- Verify's default client id scheme stops the suite checking request signatures
- HAIP track: 74 modules, all regressions today; reported as a readiness list

Runs still to do

- workflow_call from an Inji repository
- Manual fresh-clone checks on macOS and Linux
- The ten findings are drafts, not filed yet

ROADMAP

Next: file the ten findings, then close the HAIP gaps



File the ten findings

with the Inji Certify and Inji Verify teams

Add the caller workflow to Inji release pipelines

.github/examples/inji-release-caller.yml

Close the HAIP gaps

in the order of docs/certification-readiness.md

Self-certification

once the blocking findings are fixed

TEAM

Team Badmosip

KB

Kanav Bhardwaj

IB

Ivan Bhargava

NM

Nikunj Mahajan

IS

Ishaan Sodhi

HM

Harsh Modi

Repository

https://github.com/Paradox-73/Automated_Conformance_Testing_for_Inji_Certify_and_Inji_Verify

Public on GitHub, with the code, the test reports and the screenshots behind every number in this deck

TRY IT

Run it yourself

```
$  ./run-full-stack-conformance.sh --prepare-only
$  ./run-full-stack-conformance.sh --no-prepare
```

You need Docker with Compose v2 (4 GiB for Docker), bash 4.4 or newer, git. Start from a clean clone.

You see Both plans run in parallel, the HTML report is written, the stack stops, and the exit code is the gate's verdict. From a fresh clone on a Windows 11 laptop the two commands took 250 s and 728 s.

Scan for the repository

Repository **https://github.com/Paradox-73/Automated_Conformance_Testing_for_Inji_Certify_and_Inji_Verify**